Lab 4.3 · Convex Hulls in Practice

Graham scan, Jarvis march, CGAL, experiments and an application

Dr. Gildardo Sánchez Ante · Tecnológico de Monterrey · TC2038

Duration: **3 hours** · Teams of 2 · Language: C++17 (with CGAL)

Learning objectives

By the end of this lab you will be able to:

1. implement the Graham scan ($\mathcal{O}(n \log n)$) and the Jarvis march ($\mathcal{O}(nh)$) with an *exact* orientation predicate;
2. handle degenerate inputs (duplicates, collinear points, $n < 3$) according to a precise output specification;
3. validate your code against a professional library (CGAL) with randomized stress tests;
4. design and interpret running-time experiments in which the *output size* h matters;
5. use the hull to solve an application problem (perimeter, area, diameter via rotating calipers);
6. explain why naive floating-point geometric predicates are unreliable.

Schedule

Part	Activity	Time
0	Setup: install CGAL, build the starter kit	15 min
1	Exact orientation test and Graham scan	40 min
2	Jarvis march (gift wrapping)	20 min
3	Stress testing against CGAL	20 min
4	Experiments: when does h matter?	35 min
5	Application: fencing an orchard	40 min
6	Robustness: why exact predicates?	10 min
	Total	**180 min**

What you receive (starter kit)

File	Purpose
`CMakeLists.txt`	builds everything (Release mode by default)
`hull.hpp`, `hull.cpp`	**your code**: `orient`, `graham`, `jarvis`
`main_hull.cpp`	command-line driver: `./hull_main graham < input`
`cgal_hello.cpp`	Part 0: CGAL's `convex_hull_2` on the same input format
`gen.hpp`	random generators: `square`, `disk`, `parabola`
`stress.cpp`	Part 3: randomized comparison against CGAL (has TODOs)
`bench.cpp`	Part 4: timing experiments, CSV output (has TODOs)
`app.cpp`	Part 5: perimeter, area, diameter (has TODOs)
`robust.cpp`	Part 6: floating-point experiment (complete)
`hull3d.cpp`	Bonus: 3-D hull with CGAL (complete)
`data/*.txt`	test inputs (the class example and degenerate cases)

Input format. First line n; then n lines with two integers x y, where $|x|, |y| \leq 10^9$. Points may repeat.

Output convention (used by every function and by the graders).

- Only **extreme points** (corners): points lying in the middle of a hull edge are *not* reported. No duplicates.
- **Counter-clockwise** order, **starting at the lowest point** (smallest y; ties: smallest x).
- Degenerate cases: if there are 0 or 1 distinct points, output them; if all points are collinear, output the **two endpoints** of the segment (lowest first).

1 Part 0 — Setup (15 min)

Install CGAL (version 5.x or newer is header-only; it needs GMP and MPFR, which the package managers install for you):

```
# Ubuntu / Debian / WSL (recommended on Windows)
sudo apt install build-essential cmake libcgal-dev
# macOS (Homebrew)
brew install cmake cgal
# Windows without WSL: vcpkg install cgal (then pass the vcpkg toolchain file to cmake)
```

Build and run:

```
cd starter && mkdir build && cd build
cmake .. # Release mode by default
make -j4
./cgal_hello < ../data/example13.txt
```

Task 0

Run `cgal_hello` on `example13.txt`: the 13 points $p_0, \ldots, p_{12}$ from the class slides (the Jarvis and Graham animations). You should get 5 hull vertices: $(312, 56)$, $(684, 86)$, $(720, 170)$, $(420, 308)$, $(192, 146)$ — possibly starting at a different vertex.

Report question 0

CGAL does not start the hull at the lowest point. Why is a precise output convention (start point, orientation, collinear points) essential when you want to compare two implementations automatically?

2 Part 1 — The orientation test and Graham scan (40 min)

Everything in this lab rests on one predicate:

$$\text{orient}(a, b, c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x) \qquad \begin{cases} > 0 & a \to b \to c \text{ turns left (counter-clockwise)} \\ < 0 & \text{turns right} \\ = 0 & \text{collinear.} \end{cases}$$

Task 1.1 — `orient` (in `hull.hpp`)

Implement `orient` with `long long` arithmetic. **No floating point and no `atan2` anywhere in this lab.**

Report question 1.1

With $|x|, |y| \le 10^9$, prove that `orient` cannot overflow a 64-bit signed integer. What is the largest possible absolute value of each product and of the result? What would you change if coordinates could reach 10^{10}?

Task 1.2 — `graham` (in `hull.cpp`)

1. Remove duplicates; return early if fewer than 3 distinct points remain.
2. Let p_0 be the lowest point (ties: leftmost).
3. Sort the remaining points *by angle around* p_0 using a comparator built on `orient` (a comes before b if $p_0 \to a \to b$ turns left). Break ties (same direction) by distance to p_0.
4. Scan with a stack: before pushing p, pop while the top two points and p do **not** make a left turn (orient ≤ 0).

Tip

Why is the comparator in step 3 a valid strict weak ordering? Because every other point lies in the upper half-plane of p_0 (or on its right along the horizontal line), all angles are in $[0, \pi)$ — so "turns left" never wraps around. This is exactly why we start from the lowest point.

Check your code with the provided inputs:

Input file	Expected output (h, then the vertices)	What it tests
`example13.txt`	5: (312,56) (684,86) (720,170) (420,308) (192,146)	the class example
`square_degenerate.txt`	4: (0,0) (4,0) (4,4) (0,4)	duplicates, points on edges, interior
`collinear.txt`	2: (0,0) (3,3)	all points collinear, a duplicate
`single.txt`	1: (7,−2)	$n = 1$

Report question 1.2

In the class example, which points are popped from the stack, and in which order? (Compare with the animation in the slides.) Why is the scan $\mathcal{O}(n)$ even though the `while` loop can pop many points at once?

3 Part 2 — Jarvis march

(20 min)

Task 2 — `jarvis` (in `hull.cpp`)

Start at the lowest point. From the current hull vertex `cur`, take any candidate `nxt`; then for every point r, if r is to the **right** of `cur`→`nxt` (orient < 0), make r the new candidate. If r is collinear with `cur`→`nxt`, keep the *farther* point (otherwise you would report points in the middle of an edge). Stop when you return to the start. Count every call to `orient` in `*tests`.

Your Jarvis and Graham outputs must be identical on all four test files.

Report question 2

How many orientation tests does your Jarvis march perform on `example13.txt`? Express the number as a function of n and h. Which input of n points makes Jarvis march as slow as possible, and how slow?

4 Part 3 — Stress testing against CGAL (20 min)

Four hand-made tests prove little. `stress.cpp` compares your two functions with CGAL on 3000 random inputs.

Task 3

1. Complete `cgal_hull`: rotate CGAL's output so that it starts at the lowest point (`std::rotate` + `std::min_element` with our `operator<`).
2. Extend the generator loop so that some inputs contain **duplicated points** and some contain **several collinear points on a hull edge** (e.g. append points $(-1000 + 500k, -10^9)$ for $k = 0..4$: they lie on the bottom edge).
3. Run `./stress` until it prints `OK: 0 mismatches`. Fix every bug it finds.

Report question 3

List the bugs (if any) that the stress test revealed and how you fixed them. Why is it safe to convert our integer coordinates to `double` before calling CGAL? (Hint: $10^9 < 2^{53}$.) Which CGAL kernel are we using, and what does "exact predicates, inexact constructions" mean?

5 Part 4 — Experiments: when does h matter? (35 min)

Three point distributions with very different hull sizes:

`gen(...)`	Points	Expected hull size h
`square`	uniform in a square	$\Theta(\log n)$
`disk`	uniform in a disk	$\Theta(n^{1/3})$
`parabola`	on $y = x^2$	$h = n$ (every point is extreme)

Task 4

1. Complete `bench.cpp`: time your `jarvis` (also print its number of tests), `CGAL::convex_hull_2` (CGAL's default), `CGAL::ch_graham_andrew` and `CGAL::ch_jarvis`. Skip the Jarvis variants when $n \cdot h > 2 \cdot 10^9$.
2. Run `./bench > results.csv` (Release build!) and make two log–log plots (any tool: Python, Excel, gnuplot, R): (a) h vs. n for each distribution; (b) time vs. n for each algorithm, one plot per distribution.

Sanity check: with GCC (libstdc++) and the seed in `bench.cpp`, the hull sizes for $n = 10^3, 10^4, 10^5, 10^6$ are $h = 22, 26, 35, 37$ for the square and $h = 32, 73, 166, 336$ for the disk (other standard libraries may generate slightly different points). The whole benchmark should take about a minute; if your laptop is much slower, lower the $n \cdot h$ threshold.

Report questions 4

(a) From your plot (a), estimate the slope for the disk. Does it agree with $h = \Theta(n^{1/3})$?
(b) On which distribution is Jarvis march competitive with (or faster than) Graham scan? Explain using $\mathcal{O}(nh)$ vs. $\mathcal{O}(n \log n)$.
(c) On the parabola, how does the time of Jarvis march grow when n is multiplied by 10? Is that what the theory predicts?
(d) CGAL's default `convex_hull_2` uses the Akl–Toussaint heuristic: it first finds the extreme points in x and y and throws away every point inside the quadrilateral they form. Why does this help so much on the square and the disk, and not on the parabola?

6 Part 5 — Application: fencing an orchard (40 min)

An orchard has n trees at integer coordinates. The owner wants (i) the length of the shortest fence enclosing all trees, (ii) the area it encloses, and (iii) the two trees that are farthest apart (to install a sprinkler pipe between them).

Task 5 — `app.cpp`

1. `perimeter(H)`: the sum of the hull edge lengths. (Why is the shortest fence exactly the hull?)
2. `twice_area(H)`: the shoelace formula $2A = \sum_i (x_i y_{i+1} - x_{i+1} y_i)$, computed exactly with integers.
3. `diameter2(H, a, b)`: the squared diameter with **rotating calipers** in $\mathcal{O}(h)$: for each hull edge (H_i, H_{i+1}), advance a second pointer j while the distance from H_{j+1} to that edge (i.e. $\text{orient}(H_i, H_{i+1}, H_{j+1})$) keeps growing; the farthest pair is among the pairs (H_i, H_j) and (H_{i+1}, H_j). The pointer j goes around the hull only once.

Expected output for `example13.txt`:

```
h = 5
perimeter = 1224.508
area = 80316.000
diameter = 528.545 between (720, 170) and (192, 146)
```

Report questions 5

(a) Why must the farthest pair of points always be two *hull vertices*?
(b) Validate your rotating calipers against the $\mathcal{O}(n^2)$ brute force on random inputs. Then compare their running times for $n = 10^5$ points in a square. What is the total complexity of the whole pipeline (hull + calipers)?
(c) With $|x|, |y| \le 10^9$, can `twice_area` overflow a `long long`? How would you fix it? (Hint: `__int128`.)

7 Part 6 — Robustness: why exact predicates? (10 min)

Run `./robust`. It evaluates the orientation of $p = (0.5 + i\,u,\ 0.5 + j\,u)$, $q = (12, 12)$, $r = (24, 24)$ for a 256×256 grid of tiny perturbations ($u = 2^{-53}$), once with plain `double` arithmetic and once

with CGAL's exact predicate, and counts disagreements (Kettner, Mehlhorn, Pion, Schirra & Yap, "Classroom examples of robustness problems in geometric computations", 2008).

Report question 6

How many signs were wrong on your machine? Explain where the error comes from. What could happen to a Graham scan whose comparator uses such an inconsistent predicate (think about `std::sort`)? Why did our integer `orient` not have this problem?

Bonus (choose any; up to +10 points)

- **Convex layers (onion peeling).** Repeatedly compute the hull and remove its vertices. Report the number of layers (the "depth" of the set), which statisticians use to detect outliers. For `example13.txt` the layers have 5, 7 and 1 points.
- **Hulls in 3-D.** Build and run `hull3d.cpp` (CGAL's `convex_hull_3` on 10^4 random points in a ball). Verify Euler's formula $V - E + F = 2$ and the bounds $E \le 3V - 6$, $F \le 2V - 4$ from class. When do they hold with equality?
- **Chan's algorithm.** Implement the $\mathcal{O}(n \log h)$ algorithm that combines Graham scan (on groups of m points) with Jarvis march (on the group hulls), and add it to the benchmark.

Deliverables

Submit a single `.zip` (team member names in the file name) with:

1. your source code (`hull.hpp`, `hull.cpp`, `stress.cpp`, `bench.cpp`, `app.cpp`, plus any bonus files);
2. `results.csv` and your two plots;
3. a short report (PDF, 3–5 pages) answering every *Report question*, with the plots and a table of your timings.

Grading rubric (100 points)

Component	Criteria	Points
Part 1	`orient` exact; Graham scan correct on all test files and degenerate cases; Q1.1–1.2	25
Part 2	Jarvis march correct, test counter; Q2	15
Part 3	stress test completed with duplicates and collinear cases, `OK: 0 mismatches`; Q3	10
Part 4	benchmark complete, clear log–log plots, sound interpretation (Q4a–d)	20
Part 5	perimeter, exact area and $\mathcal{O}(h)$ rotating calipers correct; Q5a–c	20
Part 6	experiment run and explained	5
Code quality	readable, commented, compiles without warnings in Release mode	5
Bonus	convex layers, 3-D hull analysis, or Chan's algorithm	+10

References

- M. de Berg, O. Cheong, M. van Kreveld, M. Overmars. *Computational Geometry: Algorithms and Applications*, 3rd ed., ch. 1 and 11.
- J. O'Rourke. *Computational Geometry in C*, 2nd ed., ch. 3 (Graham scan, gift wrapping) and ch. 4.
- R. L. Graham (1972); R. A. Jarvis (1973); S. G. Akl and G. T. Toussaint (1978); T. M. Chan (1996).
- CGAL User and Reference Manual, *2D Convex Hulls and Extreme Points* and *3D Convex Hulls*: `https://doc.cgal.org`.